Introduction et généralités

Introduction.

Qu’est-ce que l’architecture du logiciel ?

- L'architecture, c'est l'art de **prendre les décisions techniques** majeures pour garantir **la pérennité et la performance d'un logiciel.**

Introduction

L'ARCHITECTURE AU SENS LITTÉRAL

Le terme **architecture** désigne la synthèse de trois piliers fondamentaux :

- Une compréhension profonde **des besoins présents et futurs** de ceux pour lequel le bâtiment est destiné et des contraintes de l'environnement,
- Une connaissance approfondie **des matériaux à assembler** ainsi que de leurs limites,
- Une connaissance approfondie **des procédés de construction** (i.e. les méthodes du génie civil, la gestion de projet, l'acquisition)

Si l'on transpose ces points au logiciel, l'architecte ne se contente pas de "coder" :

- Il comprend le besoin métier (**Business**).
- Il choisit les bonnes technologies (**Matériaux/Stacks**).
- Il définit la méthodologie de développement (**Processus/DevOps**).

Introduction

L'architecture logicielle comme le plan d'urbanisme d'une ville : elle définit comment les bâtiments (composants) sont placés, comment ils sont reliés (réseau) et comment la circulation (données) s'effectue entre eux.

Dans un système sans architecture définie, chaque nouvelle fonctionnalité augmente la complexité de manière **exponentielle**.

- **Au début :** On va vite, tout est simple.
- **Après 1 an :** Modifier une ligne de code provoque des bugs à l'autre bout de l'application.

L'architecture vise à rendre cette croissance **linéaire** plutôt qu'exponentielle en **isolant les problèmes.**

Introduction

Le piège de l'Over-engineering (sur-conception)

Attention : l'architecture peut elle-même devenir une source de complexité.

- Introduite par des choix d'outils trop lourds

(ex: utiliser 50 micro services pour un blog personnel).

- Celle qui est inhérente au problème métier

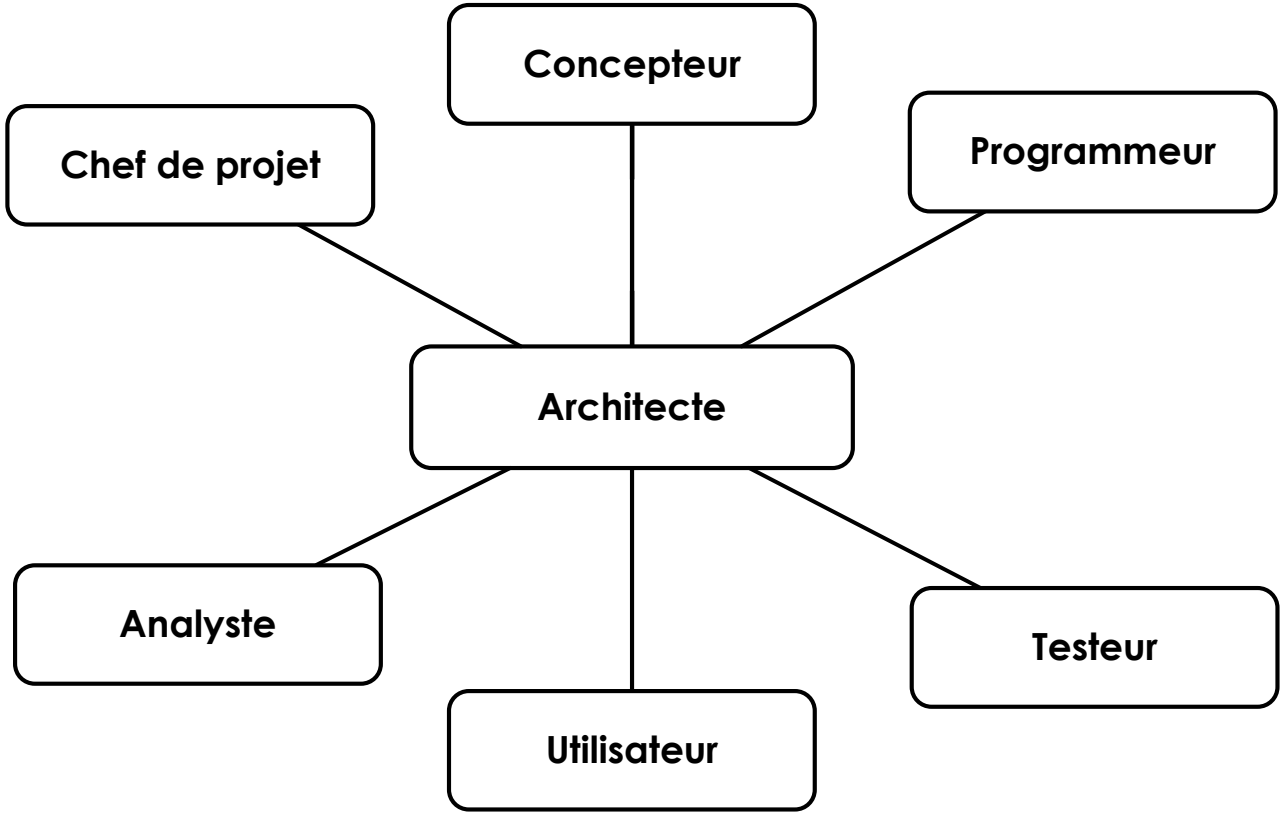
(ex: gérer des milliers de transactions bancaires par seconde).

La règle d'or : Une bonne architecture doit être **"juste assez"** complexe pour résoudre le problème, mais pas plus. **Elle doit être le remède, pas la maladie**.

Introduction

Rôle de l'architecte logiciel

- L'architecte logiciel est le pont entre la vision stratégique (le business) et la réalisation technique (le code).
- Son rôle est de prendre les décisions qui garantiront **la survie du projet à long terme.**



Introduction

ingenieur software vs architecte software

- **L'Ingénieur** est le garant de la **qualité d'exécution**.

Il s'assure que le logiciel *fonctionne* bien au quotidien.

- **L'Architecte** est le garant de la **vision stratégique**.

Il s'assure que le logiciel *survivra* aux changements futurs (augmentation du trafic, nouvelles technologies).

Les deux sont indispensables, mais pas interchangeables.

Introduction

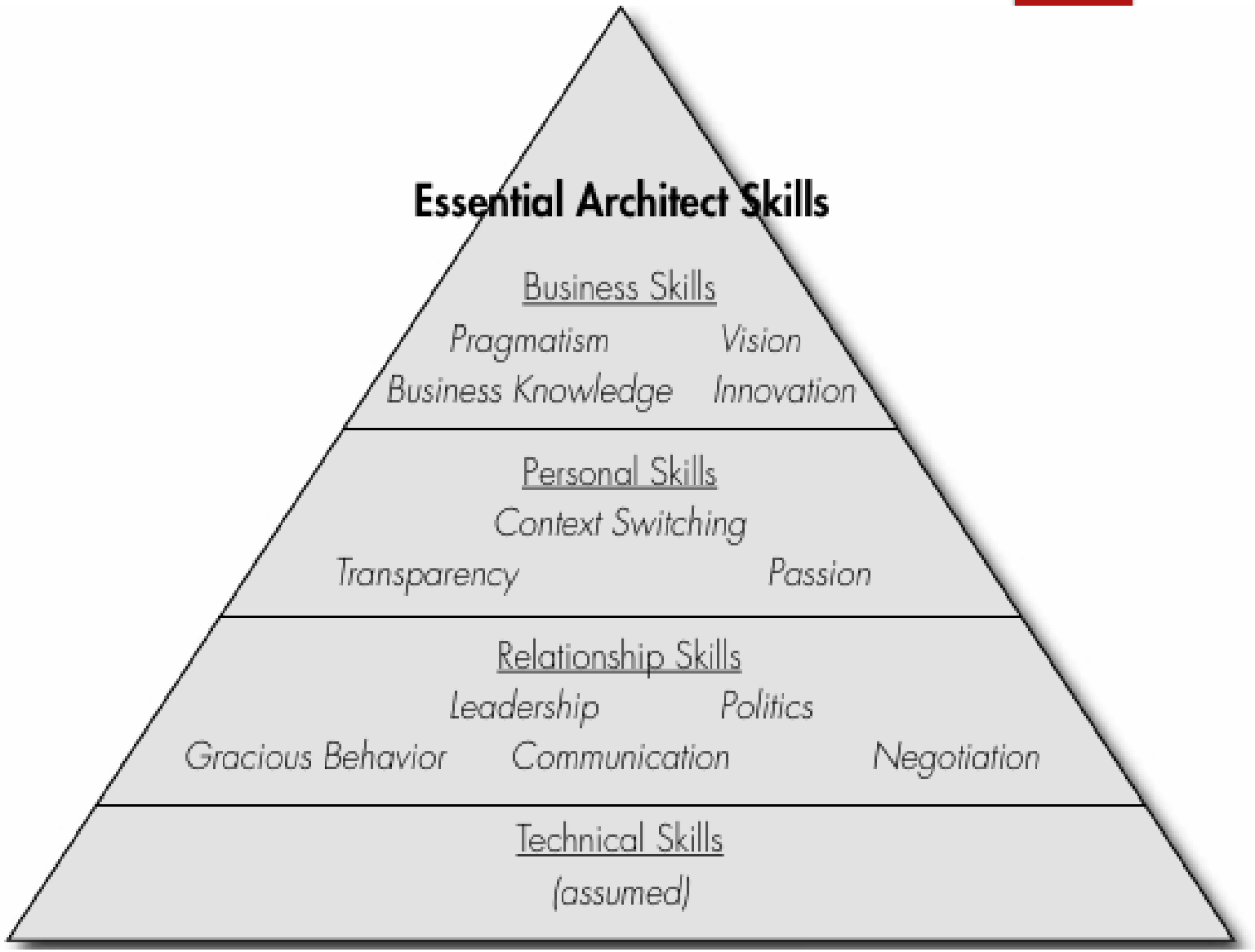
Compétences essentielles d'un architecte [i]

Compétences métiers
(Business Skills)

Compétences personnelles
(Personal Skills)

Compétences relationnelles
(Relationships Skills)

compétences techniques
(technical skills)



[i] : Hendricksen, D. (2012). ***12 Essential Skills for Software Architects*.** Addison-Wesley Professional.

Introduction

Compétences essentielles d'un architecte [i]

Compétences métiers (Business Skills)

- **Innovation** : sa capacité à proposer des idées nouvelles et innovantes pour contribuer à la croissance du chiffre d'affaires et à la pérennité de l'entreprise.
- **Connaissances métier** : sa capacité à comprendre véritablement le métier et les facteurs qui entrent en jeu lors de la prise de décisions difficiles pour assurer la croissance de l'entreprise.
- **Pragmatisme** : sa capacité à prendre les bonnes décisions en tirant profit des moyens de bord même s'ils sont minimes,
- **Vision** : sa capacité à anticiper l'évolution de l'entreprise, à contribuer à définir ses perspectives d'avenir et prévoir les changements à long terme afin d'éviter les risques les plus graves,

Compétences personnelles (Personal Skills)

- **Passion** : sa capacité à s'investir pleinement dans le projet.
- **Transparence** : sa capacité à agir et à interagir avec les partenaires, en toute transparence.
- **Adaptation au contexte** : sa capacité à s'adapter aux changements du contexte et de l'environnement.

Compétences relationnelles (Relationships Skills)

- **Négociation** : sa capacité à faire avancer les dossiers et à obtenir des résultats.
- **Communication** : sa capacité à interagir avec les partenaires et faire passer son message.
- **Comportement gracieux** : sa capacité à être poli et aimable dans toutes les situations.
- **Politique** :sa capacité à interagir dans un environnement où les intérêts peuvent diverger.
- **Leadership** : sa capacité à guider et convaincre les partenaires en expliquant le pourquoi des décisions architecturale.

[i] : Hendricksen, D. (2012). ***12 Essential Skills for Software Architects.*** Addison-Wesley Professional.

Introduction

Comment choisir la bonne architecture ?

Le choix d'une architecture doit toujours être guidé par **le contexte du projet**.

Pour orienter cette décision, plusieurs questions sont essentielles :

- **Quelle est l'ampleur du projet ?**
- **Faut-il prévoir une forte montée en charge ?**
- **Quels sont les enjeux de maintenance et d'évolution ?**
- **Quel est le niveau d'expertise de l'équipe ?**

L'architecture logicielle constitue le plus gros livrable d'un processus logiciel après le produit.

Introduction

Une architecture bien pensée, c'est avant tout :

- **De la clarté** : chaque composant a une fonction précise et une place clairement définie.
- **De la modularité** : on peut faire évoluer le système et ajouter des fonctionnalités sans tout remettre en cause.
- **De la scalabilité** : l'application encaisse la montée en charge sans perdre en stabilité.
- **De la maintenabilité** : un nouveau développeur peut rapidement comprendre l'ensemble et contribuer efficacement.

L'architecture fait toute la différence entre un projet qui se transforme en labyrinthe ingérable et **un produit capable d'évoluer sereinement**.

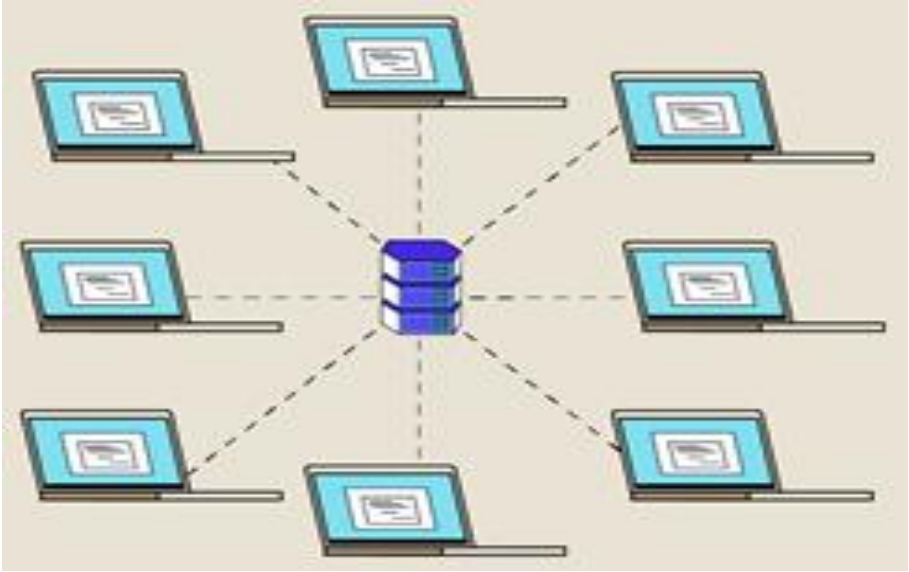
Introduction

Les architectures centralisées et les architectures distribuées

L'Architecture Centralisée

Toutes les ressources, les données et la logique de traitement résident sur un seul nœud central (serveur ou mainframe). Les terminaux des utilisateurs sont "passifs" : ils envoient des requêtes et attendent la réponse.

Exemples : Les anciens Mainframes bancaires, les applications web monolithiques classiques.



Introduction

Les architectures centralisées et les architectures distribuées

L'Architecture Centralisée

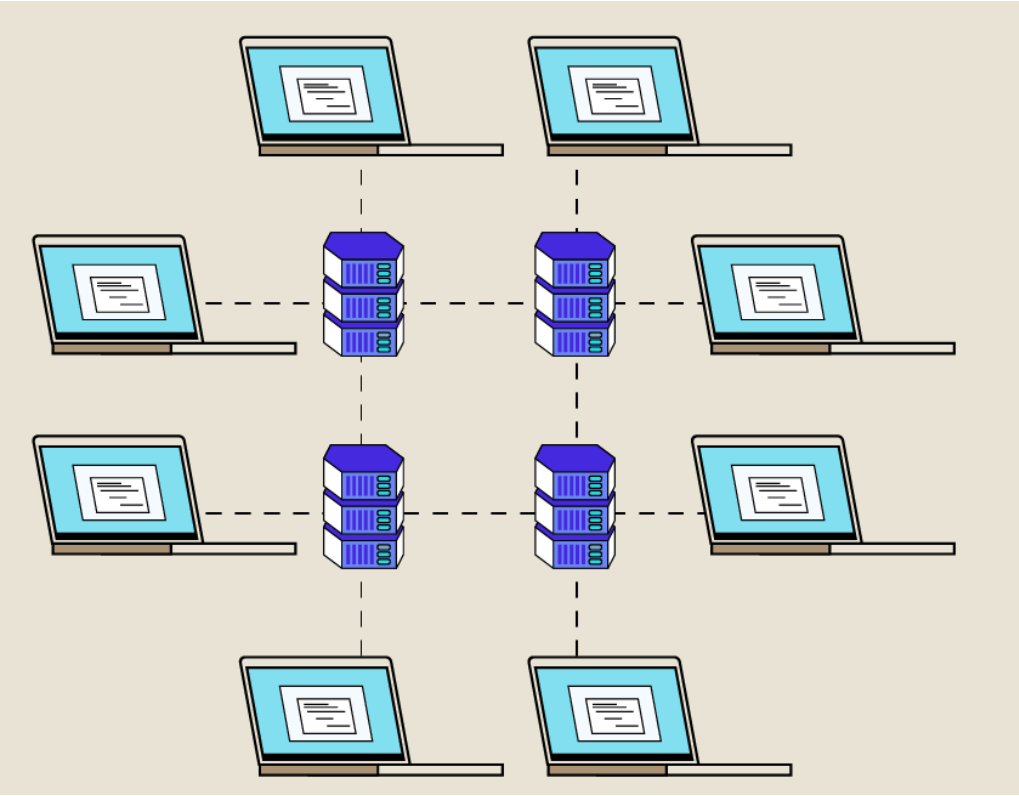
Avantages	Inconvénients
• **Simplicité de gestion :** Un seul endroit pour déployer le code, gérer la sécurité et faire les sauvegardes. • **Cohérence des données :** Pas de risque de versions divergentes des données. • **Vitesse locale :** Les appels de fonctions internes sont instantanés (pas de latence réseau entre les composants).	• **SPOF (Single Point of Failure)** Si le serveur central tombe, tout le système est mort. • **Scalabilité limitée :** Pour augmenter la puissance, il faut "scaler verticalement" (acheter un serveur plus gros/plus cher), ce qui a une limite physique. • **Goulot d'étranglement :** Toutes les requêtes arrivent au même endroit, ce qui peut saturer le système.

Les architectures centralisées et les architectures distribuées

L'architecture distribuée

Le système est composé **de plusieurs nœuds** indépendants qui communiquent entre eux via un réseau pour accomplir une tâche commune.

Exemples : Google, Netflix (Microservices), Blockchain, Réseaux P2P (BitTorrent).



Introduction

Les architectures centralisées et les architectures distribuées

L'architecture distribuée

Avantages	Inconvénients
• **Haute Disponibilité :** Si un serveur tombe, les autres peuvent prendre le relais (tolérance aux pannes). • **Scalabilité Horizontale :** Besoin de plus de puissance? On ajoute simplement de nouveaux serveurs peu coûteux. • **Flexibilité :** On peut utiliser des technologies différentes pour chaque nœud (ex: un service en Java, un autre en Python).	• **Complexité extrême :** Gérer la communication entre nœuds, la synchronisation et les pannes réseau est un défi de taille. • **Consistance des données :** il est difficile d'avoir à la fois une disponibilité totale et une cohérence immédiate des données partout. • **Latence réseau :** Faire communiquer des machines à travers le monde introduit des délais.

Introduction

Les architectures n-tiers (Architecture multi-couches)

L'architecture N-Tiers (ou multi-niveaux) est une organisation structurelle consistant à découper une application en plusieurs « étages » ou **niveaux physiques** distincts afin d'améliorer la flexibilité, la maintenance et la sécurité.

Exemple : dans une application e-commerce :

- **Présentation** → page produit affichée à l'utilisateur
- **Logique métier** → calcul du prix avec réduction
- **Données** → récupération du prix depuis la base de données

Couche Présentation
(Presentation Layer)

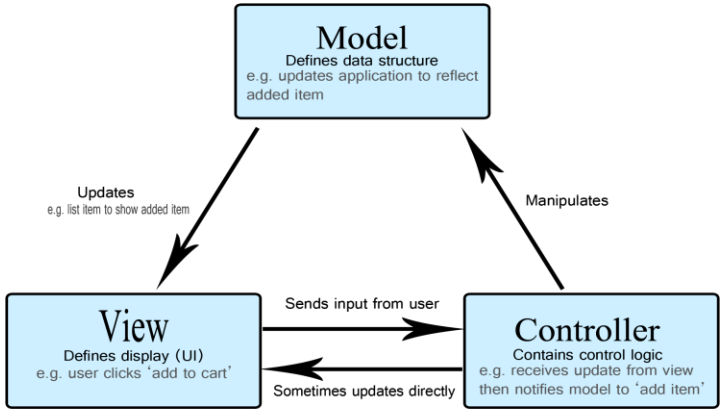
Couche Logique Métier
(Business Logic Layer)

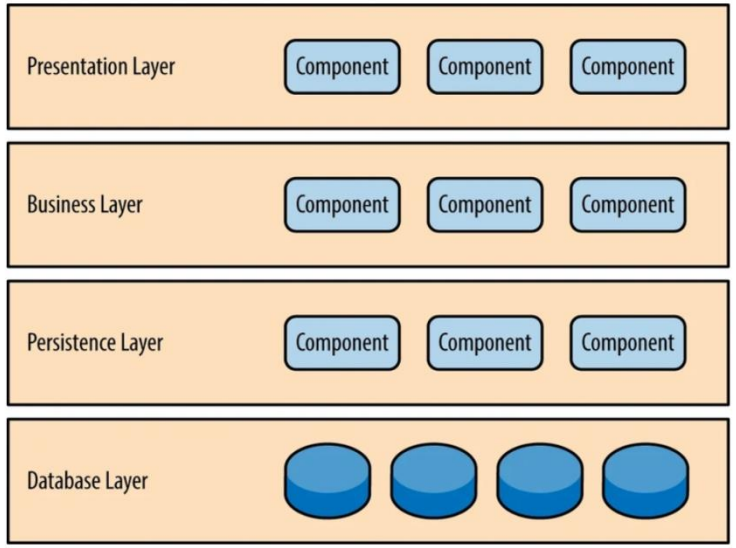
Couche Accès aux Données
(Data Access Layer)

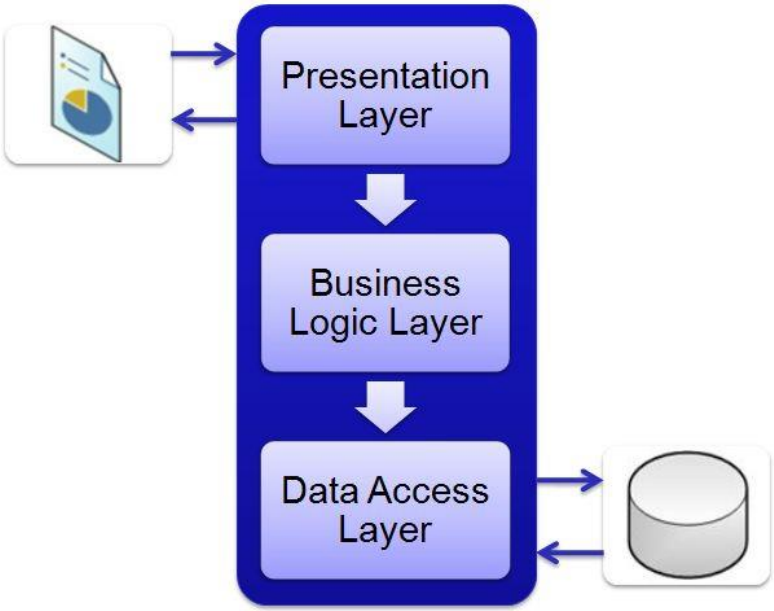
Introduction

Les modèles de structuration des systèmes et applications

Les modèles de structuration définissent **comment organiser les composants d'un système informatique** (logiciel, application web, système distribué, etc.) afin d'assurer modularité, performance, sécurité et évolutivité.







Introduction

Les modèles de structuration des systèmes et applications

Les principaux modèles utilisés en architecture logicielle :

- **Architecture Monolithique :** Toute l'application est construite en **un seul bloc**.
- **Architecture en couches (Layered / N-tiers) :** Le système est divisé en **couches hiérarchiques**, chaque couche ayant une responsabilité spécifique.
- **Architecture Client–Serveur :** Un **client** envoie des requêtes à un **serveur** qui fournit des services ou des données.
- **Architecture MVC (Modèle–Vue–Contrôleur) :** Sépare l'application en trois composants :

  Modèle (Model) → Données, **Vue (View)** → Interface utilisateur, **Contrôleur (Controller)** → Gestion des actions
- **Architecture Orientée Services (SOA) :** Le système est composé de **services indépendants** qui communiquent via un réseau.
- **Architecture Microservices :** Application divisée en **petits services autonomes**, chacun ayant sa propre base de données.
- **Architecture Orientée Événements (Event-Driven)**